

1. AWS Identity and Access Management (IAM)

- **Definition:** The AWS service for securely controlling access to AWS services and resources.
- **Practical Aspect:**
 - Created IAM Users (for human access) and IAM Roles (for AWS services/applications).
 - Attached policies (JSON documents) to grant specific permissions (e.g., AmazonS3FullAccess, AmazonEC2ReadOnlyAccess).
 - Understood the **Principle of Least Privilege**: Grant only the permissions essential to perform a task.
- **Related Concepts:**
 - **Root User vs. IAM User:** Never use the root user for daily tasks.
 - **Policies:** Define permissions. Can be AWS-managed or customer-managed.
 - **Role Assumption:** A service (like EC2) can "assume" a role to get temporary credentials.

2. AWS Security Groups

- **Definition:** A virtual firewall for your EC2 instances that controls inbound and outbound traffic.
- **Practical Aspect:**
 - Configured rules to allow SSH (Port 22) and HTTP (Port 80) traffic.
 - Created a rule with source 0.0.0.0/0 to allow public web traffic (and understood the security implications).
 - Security groups are **stateful**: If you allow an inbound request, the outbound response is automatically allowed.
- **Related Concepts:**
 - **Stateful vs. Stateless:** Network ACLs (a second layer of defense) are stateless.
 - **"Default Deny"**: All inbound traffic is denied by default; you must explicitly allow it.

3. Amazon EC2 Instances

- **Definition:** A virtual server in AWS's cloud.
- **Practical Aspect:**

- Launched an instance by selecting an **AMI** (Amazon Machine Image - like a template), an **Instance Type** (vCPUs, RAM - e.g., t2.micro), and a **Key Pair** (for SSH access).
- Connected to the instance via SSH on Linux/macOS or PuTTY on Windows using the .pem key file.
- Deployed a simple web application (like Apache) on the instance.
- **Related Concepts:**
 - **Elastic IP:** A static, public IPv4 address you can assign to an instance.
 - **Instance Store vs. EBS:** Types of storage attached to the instance. EBS is persistent.
 - **User Data:** Scripts that run automatically when the instance first launches (for bootstrapping).

4. Elastic Load Balancer (ELB)

- **Definition:** A service that automatically distributes incoming application traffic across multiple targets (e.g., EC2 instances).
- **Practical Aspect:**
 - Understood the goal: to achieve **High Availability** (service remains available even if one instance fails) and **Fault Tolerance**.
 - Traffic is distributed based on a routing algorithm (e.g., round-robin).
 - The Load Balancer has its own DNS name; clients connect to this, not directly to the instances.
- **Related Concepts:**
 - **Health Checks:** The LB regularly checks instances. If an instance fails, it stops receiving traffic.
 - **Target Groups:** A logical grouping of targets (like EC2 instances) for the LB to route to.

5. Auto Scaling

- **Definition:** Automatically adjusts the number of EC2 instances in a group based on load.
- **Practical Aspect:**
 - Created a **Launch Template** (defines what to scale - AMI, instance type, etc.).

- Set up an **Auto Scaling Group** (defines where to scale - subnets, min/max/desired capacity).
- Configured a **Scaling Policy** (defines when to scale - e.g., if CPU utilization > 70%).
- **Related Concepts:**
 - **Scale Out:** Add more instances when demand is high (triggered by a CloudWatch alarm).
 - **Scale In:** Remove instances when demand is low.
 - Works hand-in-hand with **Load Balancers**.

6. Scale Up vs. Scale Out

- **Definition:**
 - **Scale Up (Vertical Scaling):** Increasing the size of an existing instance (e.g., from t2.micro to t2.large).
 - **Scale Out (Horizontal Scaling):** Increasing the number of instances.
- **Practical Aspect:**
 - **Scale Out** is the primary, cloud-native method for handling increased load. It's done via **Auto Scaling**.
 - **Scale Up** is used for applications that cannot be distributed across multiple servers (e.g., a monolithic database). It often requires a service restart.
- **Related Concepts:**
 - **High Availability** is achieved through **Scaling Out**.
 - **Cost:** Scaling out can be more cost-effective than scaling up.

7. Jenkins / GitHub Actions

- **Definition:** Automation tools for CI/CD pipelines.
- **Practical Aspect:**
 - **GitHub Actions:** Created a workflow YAML file (.github/workflows/deploy.yml) that triggers on a git push.
 - The workflow checked out code, built a Docker image, and deployed it to an EC2 instance via SSH.
 - Understood the concept of a **Pipeline as Code**.

- **Related Concepts:**
 - **Stages:** A pipeline typically has stages like Build, Test, Deploy.
 - **Artifacts:** The output of a build stage (e.g., a JAR file, Docker image) passed to the next stage.

8. Docker & Dockerfile

- **Definition:** A platform for developing, shipping, and running applications in containers.
- **Practical Aspect:**
 - Wrote a **Dockerfile** using instructions like FROM, COPY, RUN, CMD.
 - Built a Docker image: `docker build -t my-app .`
 - Ran a container from the image: `docker run -d -p 80:80 my-app`
- **Related Concepts:**
 - **Image:** An immutable template with the application and its dependencies.
 - **Container:** A running instance of an image.
 - **Docker Hub:** A public registry to store and share images.

9. Infrastructure as Code (IaC) with Terraform

- **Definition:** Managing and provisioning infrastructure through code.
- **Practical Aspect:**
 - Wrote a .tf file using the **HashiCorp Configuration Language (HCL)**.
 - Used `terraform init` to initialize, `terraform plan` to preview, and `terraform apply` to create resources (like an S3 bucket).
 - Understood that Terraform's **state file** tracks the real-world resources it manages.
- **Related Concepts:**
 - **Declarative vs. Imperative:** Terraform is declarative (you define the *end state*), unlike scripts which are imperative (you define the *steps*).
 - **Providers:** Plugins that interact with APIs (e.g., AWS, Azure, Kubernetes).

10. Amazon S3 (Simple Storage Service)

- **Definition:** Scalable object storage for data and file hosting.

- **Practical Aspect:**
 - Created a bucket with a globally unique name.
 - Enabled **Static Website Hosting** on the bucket and uploaded index.html and error.html files.
 - Modified the bucket policy to make the objects publicly readable.
- **Related Concepts:**
 - **Object vs. Block Storage:** S3 is for objects (files), EBS is for blocks (virtual hard disks).
 - **Durability & Availability:** S3 is designed for 11 9's durability.

11. AWS Lambda & API Gateway

- **Definition:**
 - **AWS Lambda:** Serverless compute service for running code without managing servers.
 - **API Gateway:** Service to create and manage HTTP APIs.
 - **Practical Aspect:**
 - Created a simple Lambda function (e.g., in Python) that returns a JSON response.
 - Used API Gateway to create a REST API endpoint (e.g., GET /hello) that triggers the Lambda function.
 - Tested the end-to-end flow: Hitting the API URL in a browser invokes the Lambda function.
 - **Related Concepts:**
 - **Serverless:** No servers to manage; you pay only for the compute time you consume.
 - **Event-Driven Architecture:** Lambda functions are triggered by events (e.g., an HTTP request, a file upload to S3).
-